Lecture 1 — Anatomy of a Regional

> Regional Level · Prerequisites: Year Curriculum (especially Week 35, contest strategy) · Estimated time: 2 hours (1 h lecture + 1 h team triage drill)

1. Overview

An ICPC regional contest is a five-hour, team-of-three, one-computer contest with roughly 10–13 problems. By the end of the Year Curriculum you know almost every algorithm a regional requires. What separates a team that solves four problems from a team that solves seven is rarely an algorithm; it is:

1. recognising which known technique a disguised problem needs,
2. implementing it correctly on the first or second try, and
3. using three people and one keyboard efficiently for five hours.

This lecture covers the third point and sets up the rest of the track. Lectures 2–9 each attack one family of medium and medium-hard problems that show up at regionals (implementation, greedy, graph modelling, DP, sweeps, number theory, geometry, strings), and Lecture 10 is a full mock regional with a debrief.

2. The rules that shape strategy

Exact rules differ by region, so read your regional's rules page, but the ICPC scoring model is nearly universal:

- **Ranking** is by number of problems solved; ties are broken by **total penalty time**, and further ties typically by the time of the last accepted submission.
- **Penalty time** for a solved problem is the minute of the accepted submission (counted from the contest start) plus **20 minutes for every rejected submission on that problem**. Compile errors are usually not penalised. Problems you never solve contribute nothing, no matter how many wrong tries you made.
- **The scoreboard freezes** during the final hour (sometimes the final 45 minutes or less). After the freeze you still see your own verdicts but not others'.
- **One computer per team.** Three people share one keyboard and one screen; printing is usually available and fast.
- **Team reference document (notebook).** Many regionals allow a printed notebook of limited size (25 single-sided pages is common, mirroring the World Finals rule). Nothing electronic.

- **Clarifications** are requested through the judge system. Answers are often "read the statement", but sometimes they correct an error. Always check broadcast clarifications.

Two strategic consequences follow directly.

Solves dominate. An extra solved problem beats any amount of penalty. In the last hour, a 30% chance of solving one more problem is worth more than polishing anything.

Wrong answers on solved problems are expensive. Each costs as much as solving the problem twenty minutes later. Consider two teams with three solves each:

Team	Solves (minute, rejected tries)	Penalty
X	A (12, 0), B (45, 2), C (170, 0)	12 + (45 + 40) + 170 = 267
Y	A (20, 0), B (60, 0), C (110, 1)	20 + 60 + (110 + 20) = 210

Team Y wins. The 60-second pre-submission checklist (Section 6) exists to avoid team X's two careless rejections.

3. The shape of a regional problem set

Problem sets are usually **not** sorted by difficulty; many regionals (NWERC, SWERC, NAC) order problems alphabetically by title. A typical distribution:

- **2–3 easy problems** that nearly every team solves: simulation, a short formula, a direct greedy.
- **3–4 medium problems**: a known technique in disguise. BFS on a non-obvious graph, a DP with a sensible state, a greedy that needs an exchange argument, a sweep with a Fenwick tree.
- **2–3 medium-hard problems**: two techniques combined (DP plus a data structure, binary search plus flow), or a long but standard implementation (geometry, parsing).
- **2–3 hard problems** that decide first place; only a handful of teams solve them.

Medal and advancement cutoffs at most regionals fall in the medium band. If your team reliably solves all easy and most medium problems with low penalty, you are competitive almost everywhere. Hence this track's focus: medium patterns and clean execution.

4. Triage: the first twenty minutes

The goal of the first twenty minutes is to know *every* problem's rough difficulty and to get the first accepted submission.

Split the reading. With 12 problems, person 1 reads A–D, person 2 reads E–H, person 3 reads I–L. One person (often the fastest typist) first spends 2–3 minutes typing the template, compile aliases, and a test script, then starts reading their share.

Label each problem after reading it, quickly:

- **E** — I could code this right now in under 15 minutes.
- **M** — I recognise a technique or see a plausible approach, but it needs thought or careful code.
- **H** — no idea yet, or the implementation is huge.

Add a one-line idea and the key constraints (e.g. "$n \le 2 \cdot 10^5$, DP over prefix + Fenwick?").

Go. The first **E** found goes straight to the keyboard. The reader who found it codes it; the others keep reading. Do not wait for everyone to finish reading before starting to code.

The problem board

Keep one shared sheet of paper or whiteboard with a row per problem: label, owner, status (read / thinking / coding / WA-printed / AC), and the idea. It costs seconds to maintain and it prevents the two most common team failures: two people silently working on the same problem, and nobody re-reading a problem that was mislabelled. The idea column also makes hand-offs cheap.

Using the scoreboard

After about 30 minutes, the scoreboard is the best difficulty estimate you have. If many teams solved a problem you labelled H, re-read it: you probably misread something, or overestimated it. If nobody solved a problem after two hours, it is likely hard or contains a trap. Do not blindly follow the leaders, though: strong teams sometimes solve a hard problem early because one member has seen it before.

5. Three people, one computer

The keyboard is the bottleneck resource. The goal is that the machine is almost always typing a solution that has already been designed.

- **The keyboard is not for thinking.** If you sit down and stare, get up. Design on paper first, including the data layout and tricky loops.
- **On a wrong answer, print and leave.** Debugging on paper while someone else types the next problem is far more efficient than debugging at the screen.
- **Announce hand-offs.** "I need the machine for about ten minutes for D, after Ana's submission."

- **Design by two, code by one.** For medium problems, the owner explains the idea to a teammate in two minutes before coding. The listener tries small and extreme cases. This catches a large fraction of wrong ideas before any code is written.

A useful default for a 5-hour plan:

Time	Focus
0:00–0:20	Read everything, label, first easy AC
0:20–1:30	Finish all easies; mediums designed in parallel
1:30–3:30	Mediums; one person scouts a medium-hard
3:30–4:00	Pre-freeze stand-up: realistic remaining targets
4:00–5:00	Commit to 1–2 problems, pair on them

Hold a two-minute stand-up around the board every hour: what is solved, what is in progress, what is the next target.

When to switch

- Stuck for 30–40 minutes with no new idea: swap problems with a teammate. Fresh eyes on a problem are cheap.
- An unexplained WA for 20 minutes: write a brute force and stress test when the machine is free, or hand the printout to a teammate.
- In the last hour, switch only towards a problem for which someone has a concrete plan.

Roles

Teams usually settle into roles such as implementer (fast, accurate code, simulation, data structures), mathematician (number theory, combinatorics, geometry), and captain/generalist (tracks the board, reads the scoreboard, graphs and DP). Treat roles as defaults. Everyone must be able to type any notebook snippet.

6. Avoiding penalties

The pre-submission checklist

Before every submission, spend one minute on:

1. All samples pass, with the exact output format.
2. Overflow: any sum, product, or count above about $2 \cdot 10^9$ needs `long long` (or `__int128`).
3. Extremes: n = 1, n at maximum, all elements equal, answer zero or "impossible".
4. Complexity at the *maximum* constraints, not the sample size.

5. Output details: precision, capitalisation, separators, a trailing newline.
6. Debug output removed (or sent to `cerr`, which judges ignore).

After a rejection

- Wrong answer: re-read the statement first. Misreading is the single most common cause. Then check overflow, limits, output format, and hand-made corner cases.
- Time limit: recompute complexity with the true limits. Check for `endl` in loops, missing fast I/O, accidentally quadratic `string` concatenation, or `map` where an array suffices.
- Runtime error: out-of-bounds indices, recursion depth (deep DFS on a path of 10^6 nodes), division by zero, uninitialised variables.

Never resubmit unchanged code hoping the judge was wrong.

Stress testing

The most powerful debugging tool you have is comparing a solution against a brute force on thousands of small random inputs. At a regional, it costs about ten minutes of machine time. Keep this loop in your notebook:

```
# gen.py prints a random small test (seeded by $1); brute is obviously correct
for i in $(seq 1 500); do
  python3 gen.py $i > in
  ./sol < in > o1
  ./brute < in > o2
  cmp -s o1 o2 || { echo "mismatch on test $i"; cat in; break; }
done
```

7. The team notebook

A good notebook contains only code you have compiled and verified on a real problem, organised by topic with a one-page index and the complexity of each snippet. A reasonable regional notebook includes a template, DSU, segment tree with lazy propagation, Fenwick tree, Dijkstra, Dinic, LCA by binary lifting, SCC, KMP / Z / polynomial hashing, Manacher, a geometry core (point struct, cross product, segment intersection, convex hull, point in polygon), extended Euclid and CRT, modular combinatorics, and Miller–Rabin with Pollard's rho. Every lecture in this track ends with code that is a candidate for your notebook.

8. Reference implementations

8.1 Contest template

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define all(x) (x).begin(), (x).end()

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    // read input, solve, print
    return 0;
}
```

8.2 An ICPC scoreboard

This program implements the scoring rules above. It is a good warm-up because it exercises exactly the things that cost penalty in contests: a custom comparator, careful tie handling, and ignoring irrelevant events (submissions after an accepted one, compile errors).

Input: `T P S`, then `S` lines `minute team problem verdict`, sorted by minute, with teams numbered `1..T`, problems given as letters, and verdict `AC`, `CE`, or anything else for a rejection. Output: rank, team, solved, and penalty, where equal (solved, penalty) pairs share a rank.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int T, P, S;
    cin >> T >> P >> S;
    vector<int> solved(T + 1, 0);
    vector<long long> penalty(T + 1, 0);
    vector<vector<int>> wrong(T + 1, vector<int>(P, 0));
    vector<vector<bool>> done(T + 1, vector<bool>(P, false));
    for (int i = 0; i < S; i++) {
        int minute, team; char prob; string verdict;
        cin >> minute >> team >> prob >> verdict;
        int p = prob - 'A';
        if (done[team][p]) continue;              // submissions after AC are ignored
        if (verdict == "AC") {
            done[team][p] = true;
            solved[team]++;
            penalty[team] += minute + 20LL * wrong[team][p];
        } else if (verdict != "CE") {             // compile errors are usually free
            wrong[team][p]++;
        }
    }
```

```
    vector<int> order(T);
    iota(order.begin(), order.end(), 1);
    sort(order.begin(), order.end(), [&](int a, int b) {
        if (solved[a] != solved[b]) return solved[a] > solved[b];
        if (penalty[a] != penalty[b]) return penalty[a] < penalty[b];
        return a < b;
    });
    int rank = 0;
    for (int i = 0; i < T; i++) {
        int t = order[i];
        if (i == 0 || solved[t] != solved[order[i - 1]] || penalty[t] != penalty[order[i
- 1]])
            rank = i + 1;
        cout << rank << ' ' << t << ' ' << solved[t] << ' ' << penalty[t] << '\n';
    }
}
```

Sample input and output:

```
3 3 7
10 1 A AC
15 2 A WA
22 2 A AC
40 3 B AC
41 1 B WA
90 1 B AC
100 3 C CE
```

```
1 1 2 120
2 3 1 40
3 2 1 42
```

Team 1 solved A at minute 10 and B at minute 90 after one rejection, so its penalty is 10 + 90 + 20 = 120.

9. Common mistakes

- **Reading in lock-step.** All three people read problem A first. Split the stack.
- **Labelling once and never revisiting.** A problem labelled H in minute 10 may be easy once you see 40 teams solved it.
- **Thinking at the keyboard.** The machine sits idle while one person ponders; two others wait.
- **Debugging on screen.** Print the code. Let the next solution be typed.
- **Unchecked submissions.** Overflow, wrong output format, and forgotten debug output account for many 20-minute penalties.
- **Last-hour chaos.** Three half-written solutions and none accepted. Commit to at most two targets after the pre-freeze stand-up.

- **Untested notebook code.** A snippet that has never been run is a liability, not an asset.

10. Practice problems

Warm-ups (aim for under 10 minutes each, including reading):

- Kattis `different` — A Different Problem (watch for 64-bit values): https://open.kattis.com/problems/different
- Kattis `pot` — Pot: https://open.kattis.com/problems/pot
- Kattis `carrots` — Solving for Carrots: https://open.kattis.com/problems/carrots
- Kattis `quadrant` — Quadrant Selection: https://open.kattis.com/problems/quadrant
- Codeforces 4C — Registration System: https://codeforces.com/problemset/problem/4/C
- Codeforces 1B — Spreadsheets: https://codeforces.com/problemset/problem/1/B
- CSES 1619 — Restaurant Customers: https://cses.fi/problemset/task/1619

Team drills:

- Pick a recent NWERC, SWERC, NAC, or CERC problem set (most are mirrored in Codeforces Gym and many on Kattis). Do it as a 5-hour virtual contest with one computer. Afterwards compare your solve order with the actual scoreboard.
- **Triage-only drill:** give the team 20 minutes with an unseen regional set, label every problem E/M/H with a one-line idea, then compare with the official scoreboard's solve counts. Repeat until your labels correlate well with reality.

11. Further reading

- ICPC rules and regional pages: https://icpc.global
- Steven Halim, Felix Halim, Suhendry Effendy — *Competitive Programming 4*, Chapter 1 (contest tips and team strategy).
- Antti Laaksonen — *Competitive Programmer's Handbook* (free PDF from https://cses.fi/book/), for quick refreshers on any Year Curriculum topic.
- KACTL, the KTH team notebook (https://github.com/kth-competitive-programming/kactl) — a model of a well-tested, compact team reference document.
- Year Curriculum Week 35 (contest strategy, stress testing) and Week 36 (final review) in this course.